

Note: Shared Key Recovery Attack on Cascader Key Exchange Protocol

Nick Aquina , Simon Rommel  and Idelfonso Tafur Monroy 

Eindhoven University of Technology, Department of Electrical Engineering, Eindhoven, The Netherlands

Abstract. Cascader has been introduced as a new key exchange protocol based on iterative multiplicative recurrence. This short note presents a practical shared key recovery attack on the Cascader key exchange protocol. This note also shows that Cascader as a hash function is not collision resistant, presents a new upper bound on the output space of Cascader and shows that a Cascader-based KDF is not secure against an Adaptive Chosen Public Inputs Attack (CPM).

Keywords: Cascader · key exchange protocol

1 Introduction

Cascader has been introduced as a new key exchange protocol based on iterative multiplicative recurrence [Lin25]. In the Cascader protocol, Alice and Bob randomly generate 256-bit private keys, a and b , respectively. Let $\mathcal{I}(e)$ be the set of all the bit indexes of e where the bit is 1:

$$\mathcal{I}(e) = \{j \in 1, 2, 3, \dots, 256 \mid e_{j-1} = 1\}$$

Alice and Bob then calculate $P(a)$ and $P(b)$, respectively, as follows:

$$P(e) = \prod_{j \in \mathcal{I}(e)} \left(3^j \cdot 2^{\frac{j(j-1)}{2}} \right) \mod p, \quad \text{where } p = 2^{256} - 189$$

Alice and Bob then calculate their public keys, A and B , respectively, as follows:

$$E = F(\text{SEED}, e) = \text{SEED} \cdot P(e) \mod p, \quad \text{where } \text{SEED} = \frac{2^{256}}{5}$$

They then proceed to publicly share their public key. Alice arrives at the shared secret S by calculating:

$$S = B \cdot P(a) \mod p$$

and Bob by calculating:

$$S = A \cdot P(b) \mod p$$

They arrive at the same shared secret since:

$$B \cdot P(a) = \text{SEED} \cdot P(b) \cdot P(a) = A \cdot P(b) \mod p$$

The authors would like to acknowledge the funding received under the FIQCS project (NWA.1436.20.005).

E-mail: n.aquina@tue.nl (Nick Aquina), s.rommel@tue.nl (Simon Rommel), i.tafur.monroy@tue.nl (Idelfonso Tafur Monroy)

This work is licensed under a “CC BY 4.0” license.

Date of this document: 2025-08-04.



2 Shared Key Recovery Attack

In a secure key exchange protocol it should be hard for an attacker to recover the shared secret, this implies that it should also be hard to recover the private key from the public key. Although we have not found a way to recover the private key from the public key, we do show that the shared secret between Alice and Bob can be recovered using only public values. Since Alice calculates the shared secret as $S = B \cdot P(a)$, knowledge of a is not necessary when $P(a)$ is known. $P(a)$ can be recovered from the public key A by using the modular multiplicative inverse of SEED, which can be calculated using the extended Euclidean algorithm. The shared secret can thus be calculated as follows:

$$\begin{aligned} P(a) &= A \cdot \text{SEED}^{-1} \mod p \\ S &= B \cdot P(a) \mod p \end{aligned}$$

Appending the code in listing 1 after the reference implementation from [Lin25] will print the shared secret between Alice and Bob.

```
function modInv(a, b) {
  return 1n < a ? b - modInv(b % a, a) * b / a : 1n;
}
console.log(bobPublic * modInv(SEED, MOD) * alicePublic % MOD)
```

Listing 1: Shared key recovery attack against the reference implementation using the multiplicative modular inverse of SEED [Dav16].

3 Cascader as hash function

One of the open questions presented in the paper introducing Cascader questions if Cascader can be used as a hash chain. We show that the Cascader hash function as defined in definition 1 is not collision resistant.

Definition 1. $\text{HASH}(m) = F(\text{SEED}, m)$ is the hash function for Cascader with a fixed input length of 256 bits, a fixed output length of 256 bits, in which m is the message.

A secure hash function should have collisions resistance, i.e. it should be hard for an attacker to find two messages that hash to the same value [RS04]. For all messages m between 0 and 2^{20} (i.e. 1 048 576), only 42 891 unique hash values exist, i.e. almost 96% of the hash values collide. For example, the first collision is $F(\text{SEED}, 24) = F(\text{SEED}, 35)$:

$$\begin{aligned} P(24) &= 3^4 \cdot 3^5 \cdot 2^6 \cdot 2^{10} = 3^9 \cdot 2^{16} \\ P(35) &= 3^1 \cdot 3^2 \cdot 3^6 \cdot 2^0 \cdot 2^1 \cdot 2^{15} = 3^9 \cdot 2^{16} \end{aligned}$$

From this example it follows that for any message with the least significant bits 011000 (24), a collision can be found by replacing those bits with 100011 (35).

4 Upper bound on the output space of Cascader

Another open question presented in the paper introducing Cascader questions what the coverage of the output space of Cascader is. We show that it is lower than approximately 2^{72} .

Any output of Cascader is of the form:

$$F(\text{SEED}, e) = \text{SEED} \cdot 3^x \cdot 2^y \mod p$$

As x is the summation of values between 1 and 256, the maximum value for x is $\frac{(256+1) \cdot 256}{2} \approx 2^{15}$. As y is the summation of values between 0 and $\frac{2^{15} \cdot (2^{15}-1)}{2} \approx 2^{29}$, the maximum value for y is $\frac{(2^{29}+1) \cdot 2^{29}}{2} \approx 2^{57}$. The maximum number of possible output values is then $2^{15} \cdot 2^{57} \approx 2^{72}$. This is an upper bound, the actual number of possible output values is lower as not all possible combinations of x and y are valid for Cascader. Thus, there are at least $2^{256} - 2^{72} \approx 2^{256}$ collisions.

5 Cascader as KDF

Another open question presented in the paper introducing Cascader questions if Cascader can be used as a slow KDF. We show that the construction of Cascader as a KDF as defined in definition 2 is not CPM-secure, i.e. not secure against an Adaptive Chosen Public Inputs Attack (CPM) [WDGS12].

Definition 2. $\text{KDF}(\sigma, r, \text{null}, 256) = F(r, \sigma)$ is the KDF for Cascader with a fixed output length of 256 bits and no context information in which σ is the keying material and r the salt.

In the CPM-game for the Cascader KDF, an attacker chooses the salt r and receives either the output of $F(r, \sigma)$ or a random string of 256 bits and wins if he can correctly guess which of the two possibilities he received. The KDF is secure if no attacker can win the game with a probability greater than $\frac{1}{2} + \epsilon$. However, if an attacker chooses the salt 0, then the output will always be $F(0, \sigma) = 0 \cdot P(\sigma) \mod p = 0$ and the attacker will know that he received the output of the KDF in case he received 0. The attacker will always win this game unless the random bit string 0 was chosen. The probability of the attacker winning is therefore $1 - \frac{1}{2^{256}} \approx 1$.

References

- [Dav16] David Wörn. Short modular inverse. Codeforces. February 8, 2016. URL: <https://codeforces.com/blog/entry/23365>.
- [Lin25] Anders Lindman. Cascader: a recurrence-based key exchange protocol. Cryptology ePrint Archive, Paper 2025/1304, 2025. URL: <https://eprint.iacr.org/2025/1304>.
- [RS04] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In Bimal K. Roy and Willi Meier, editors, *FSE 2004*, volume 3017 of *LNCS*, pages 371–388. Springer, Berlin, Heidelberg, February 2004. DOI: [10.1007/978-3-540-25937-4_24](https://doi.org/10.1007/978-3-540-25937-4_24).
- [WDGS12] Chuah Chai Wen, Edward Dawson, Juan Manuel González Nieto, and Leonie Simpson. A framework for security analysis of key derivation functions. In *Information Security Practice and Experience*. Volume 7232, pages 199–216. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. DOI: [10.1007/978-3-642-29101-2_14](https://doi.org/10.1007/978-3-642-29101-2_14).